

Design layer

the whole layer, as one document

Contents

1 AgentObs — draft design	4
1.1 Foundation	4
1.2 Pending updates	4
1.3 System at a glance	5
1.4 Instrumentation core	6
1.5 Handler pipeline	8
1.6 Framework adapters	9
1.7 End-to-end walkthrough	10
Glossary	10

ROOT DESIGN

AgentObs — draft design

1 AgentObs — draft design

1.1 Foundation

This draft preserves the legacy design while separating confirmed implementation from unresolved intent, as recorded by [ADR-0006](#).

GOAL

Instrument LLM agent operations without backend coupling

composition

NO-GOAL

Couple instrumentation helpers to one observability backend

INVARIANT

Instrumentation preserves the wrapped return value

A valid [Instrumentation helper](#) returns the wrapped operation's result shape after adding telemetry metadata around it.

invariants

mechanism

INVARIANT

Ordinary child completion restores its parent

On the normal completion path, a handler restores the parent [Span context](#) after ending a nested operation of a different event type, as recorded by [ADR-0004](#). Same-type nesting and callback-failure cleanup remain unresolved.

state

mechanism

PRINCIPLE

Translate after the backend-neutral boundary

composition

1.2 Pending updates

foundation

Approve the migrated foundation

2026-09-02

The goal, no-goal, and principle were recovered from the legacy design and require owner approval.

ruling

Choose failure-event semantics

2026-09-02

The legacy design requires exception events, while the implementation converts failures into stop events with error metadata. See [issue 01](#).

ruling

Choose runtime configuration semantics

2026-09-02

Runtime environment updates do not restart handlers or prevent helpers from emitting events. See [issue 02](#).

ruling

Choose exporter configuration ownership

2026-09-02

Handler-specific endpoint settings are documented but the global OpenTelemetry exporter owns transport. See [issue 03](#).

ruling

Choose Generic handler span kinds

2026-09-02

The documentation promises standard span kinds but the implementation leaves the OpenTelemetry kind at its default. See [issue 04](#).

ruling

Choose streaming completion semantics

2026-09-02

The legacy design calls ReqLLM streaming non-blocking, while the adapter consumes the source stream before returning a replay stream. See [issue 07](#).

ruling Choose LLM stop validation

2026-09-02

The legacy schema requires chat output messages, while the current validator accepts every LLM stop map. See [issue 08](#).

ruling Choose the Jido tracer backend boundary

2026-09-02

The broad backend-neutral goal and the direct OpenTelemetry Jido integration need an owner-defined boundary. See [issue 09](#).

ruling Choose span-context lifecycle semantics

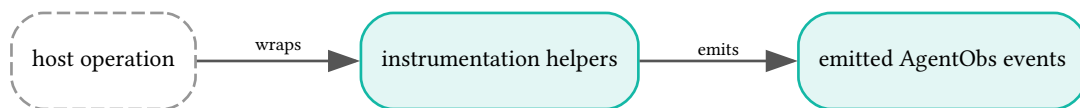
2026-09-02

Same-type nesting can overwrite an outer handler context, and callback failures can skip restoration. See [issue 10](#).

1.3 System at a glance

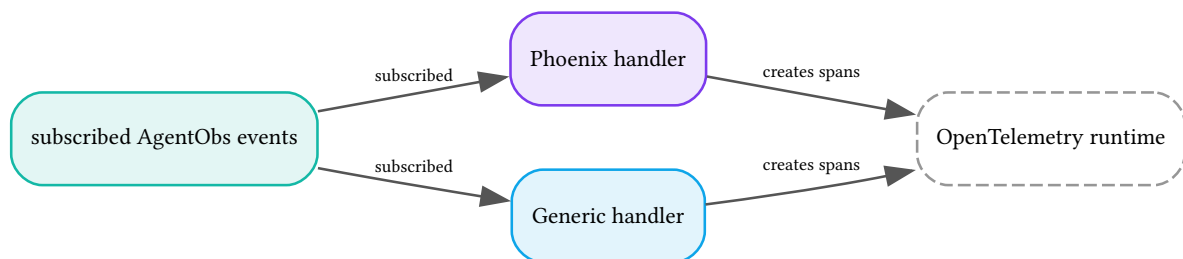
AgentObs is one instrumentation context with optional framework adapters and two trace translations.

ALTITUDE L2 · CONTEXTS calls become events and spans



The **ReqLLM adapter** enters through the public helpers. The **Jido tracer** is a separate adapter that creates OpenTelemetry spans from Jido callbacks without emitting AgentObs events.

ALTITUDE L2 · CONTEXTS events fan out to configured trace translations



The system has no durable domain entities because it transforms application calls into telemetry and trace data. **ADR-0005** records this modeling boundary.

1.4 Instrumentation core

The core validates operation metadata, executes application code, and emits the backend-neutral lifecycle chosen in [ADR-0001](#).

Public instrumentation API

Wrap the four validated operation lifecycles and emit discrete custom events.

responsibility	Merge operation identity into caller metadata and invoke wrapped functions; <code>emit/2</code> sends a custom event outside those four span lifecycles.
interface	<code>trace_agent/3</code> , <code>trace_tool/3</code> , <code>trace_llm/3</code> , <code>trace_prompt/3</code> , <code>emit/2</code> , and <code>configure/1</code> .
interactions	Valid helpers call the event schema before <code>:telemetry.span/3</code> ; custom emission calls <code>:telemetry.execute/3</code> .
invariants	The wrapped return shape is preserved for successful, error, and unrecognized values.
failure	Invalid start metadata logs a warning and runs the function without span events. Raised, thrown, and exited failures are currently converted to error values.

Event schema

Define the metadata contract shared by emitters and handlers.

responsibility	Validate required start and stop fields and normalize LLM message roles.
interface	<code>validate_event/3</code> , <code>normalize_metadata/3</code> , <code>event_types/0</code> , and <code>event_phases/0</code> .
interactions	Instrumentation helpers validate before emission; translators consume the emitted metadata.
invariants	Supported operation types are agent, tool, LLM, and prompt; supported phases are start, stop, and exception.
failure	Unsupported types, phases, or required fields return a descriptive error tuple.

Agent event contract

Describe an agent loop or invocation.

responsibility	Carry agent identity, input, output, and optional execution context.
interface	<code>[:prefix, :agent, :start :stop :exception]</code> .
interactions	Start requires name and input; it may include <code>model</code> , <code>session_id</code> , <code>user_id</code> , and custom metadata. Stop requires output; it may include <code>tools</code> , <code>iterations</code> , <code>tokens</code> , <code>cost</code> , and custom metadata.
invariants	The configured event prefix precedes the operation type and lifecycle phase.
failure	Exception metadata is accepted without required fields; helper-captured failures currently reach stop metadata instead.

Tool event contract

Describe one tool execution.

responsibility	Carry the tool identity, arguments, optional description, and result.
interface	<code>[:prefix, :tool, :start :stop :exception]</code> .
interactions	Start requires name and arguments; stop requires <code>result</code> . Arguments may be a map or encoded string.
invariants	Tool inputs and outputs remain backend-neutral event values until a handler translates them.
failure	Exception metadata has no required fields; invalid start or stop metadata prevents instrumentation but not execution.

LLM event contract

Describe a chat, completion, or embedding request.

responsibility	Carry model inputs, invocation parameters, outputs, token use, cost, and finish reason.
interface	<code>[:prefix, :llm, :start :stop :exception]</code> .
interactions	Start requires model; chat is the default type and also requires <code>input_messages</code> . Optional request fields include type, temperature, maximum tokens, top-p, top-k, penalties, and custom metadata. Stop may carry output messages, prompt/completion/total tokens, cost, and finish reason.
invariants	Message roles expressed as atoms normalize to strings; valid explicit types are chat, completion, and embedding.
failure	The current validator accepts any stop map, while the legacy contract requires chat output messages; issue 08 holds the ruling.

Prompt event contract

Describe prompt-template rendering.

responsibility	Carry a template name, input variables, optional template text, and rendered output.
interface	<code>[:prefix, :prompt, :start :stop :exception]</code> .
interactions	Start requires name and variables; stop requires rendered.
invariants	Prompt rendering remains a validated lifecycle even though Phoenix translates its span kind as CHAIN.
failure	Exception metadata has no required fields; missing required metadata returns a descriptive validation error.

BEHAVIOR

Valid instrumentation emits one completed lifecycle

- Given** The start metadata satisfies the event schema.
- When** A caller executes an instrumentation helper.
- Then** A start event precedes the wrapped operation.
- Then** A stop event follows the wrapped operation with derived result metadata.
- Then** The caller receives the wrapped operation's result shape.

level: boundary area: Valid public instrumentation

BEHAVIOR

Invalid metadata leaves the operation uninstrumented

- Given** The start metadata does not satisfy the event schema.
- When** A caller invokes a helper with invalid metadata.
- Then** The operation runs once without lifecycle events.
- Then** A warning describes the validation failure.

level: boundary area: Invalid public instrumentation

1.5 Handler pipeline

Supervised handlers subscribe synchronously and maintain nested span context in the emitting process, following [ADR-0002](#).

Handler supervision

Start each configured backend subscriber independently.

responsibility	Start configured handler modules under a one-for-one supervisor when the application is enabled.
interface	The application environment supplies enabled, handlers, the event prefix, and handler-specific maps.
interactions	The application starts the supervisor; the supervisor calls each handler's child specification.
invariants	One handler restart does not restart its peers.
failure	A handler that cannot attach fails its child start and follows ordinary supervisor restart behavior.

Phoenix handler and translator

Create OpenInference-compatible spans from AgentObs events.

responsibility	Translate event metadata to OpenInference attributes and manage the corresponding span lifecycle.
interface	The handler callbacks attach, handle an event, and detach; the translator maps start, stop, and exception metadata.
interactions	Observed implementation: the handler calls the pure translator and the host-configured OpenTelemetry runtime. Handler-specific exporter settings remain an unresolved legacy contract.
invariants	On normal completion, nesting across different event types restores the parent context; each event type uses one process-dictionary key.
failure	Unexpected handler errors are logged and suppressed so telemetry callbacks return :ok; missing active spans produce warnings. Same-type nesting and restoration after callback failures remain pending.

Generic handler

Create backend-neutral OpenTelemetry spans with simplified attributes.

responsibility	Map AgentObs metadata to basic input, output, duration, token, and cost attributes.
interface	The same handler callbacks and event subscription surface as the Phoenix handler.
interactions	Observed implementation: it writes spans through the host-configured OpenTelemetry runtime without the Phoenix translator. Exporter ownership remains pending.
invariants	The ordinary completion path reinstates a parent when nested operations use distinct event types; the handler holds one process-dictionary slot for each type.
failure	Encoding falls back to inspected text; callback errors are logged and suppressed; missing active spans produce warnings. Recursive same-type nesting and cleanup after callback failure remain pending.

OBSERVED PHOENIX ATTRIBUTE MAPPING

- Agent start maps to `openinference.span.kind=AGENT`, `input.value`, text MIME type, optional `llm.model_name`, session/user fields, and flattened custom metadata. Agent stop maps `output.value`, indexed tools, iterations, token counts, cost, and latency.
- Tool start maps to `openinference.span.kind=TOOL`, `tool.name`, `tool.description`, `input.value`, and encoded `tool.parameters`. Tool stop maps `output.value` and latency.
- LLM start maps to `openinference.span.kind=LLM`, `llm.model_name`, provider fields, invocation parameters, AI SDK identifiers, and flattened input messages. LLM stop maps flattened output messages, `llm.token_count.prompt`, `llm.token_count.completion`, total tokens, `llm.cost.total`, finish reasons, `output.value`, and latency.
- Prompt start maps to `openinference.span.kind=CHAIN`, encoded variables, and optional template fields. Prompt stop maps rendered output and latency.
- A message at index `N` uses `llm.input_messages.N.message.role` or the corresponding output path. A tool call at index `M` appends `.message.tool_calls.M.tool_call.function.name` and `.function.arguments`.
- Exception translation maps type, message, escaped status, optional stacktrace, and latency; whether helpers emit exception events remains pending.

The `Handler` callback runs synchronously in the process emitting the AgentObs event. Expensive translation or export work therefore affects that process until the OpenTelemetry runtime accepts the span.

1.6 Framework adapters

Optional adapters keep framework-specific shapes outside the core event contract; `ADR-0003` records the ReqLLM boundary choice.

ReqLLM adapter

Instrument text, object, stream, and tool operations exposed by ReqLLM.

responsibility	Derive messages, usage, finish reason, objects, and tool calls from ReqLLM results.
interface	Text and object generation wrappers, streaming wrappers, tool execution, and stream collection helpers.
interactions	It calls ReqLLM and routes derived metadata through the public AgentObs helpers.
invariants	The returned response remains a ReqLLM response shape and collected usage defaults missing token counts to zero.
failure	Non-bang calls return ReqLLM error tuples; bang calls raise; malformed tool-argument fragments become an empty map.

Jido tracer

Translate Jido composer callbacks into nested OpenTelemetry spans.

responsibility	Classify composer agent, LLM, tool, and iteration callbacks and translate their metadata.
interface	<code>span_start/2</code> , <code>span_stop/2</code> , and <code>span_exception/4</code> implement the Jido tracer behaviour.
interactions	It uses the Phoenix translator and OpenTelemetry directly rather than the AgentObs event bus.
invariants	The returned tracer context contains the span and parent context required to finish or restore the trace.
failure	Nil span contexts are accepted as no-ops; exception callbacks record the failure and restore the parent context only when the callback reaches its normal cleanup path.

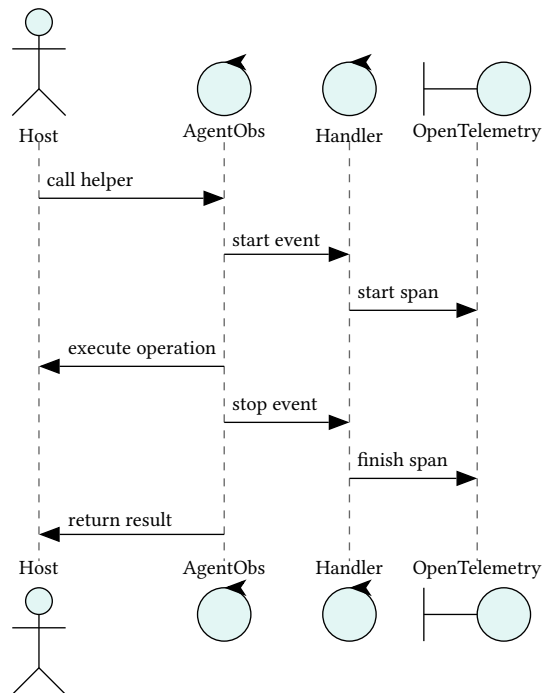
OBSERVED STREAMING BEHAVIOR

The current ReqLLM streaming wrappers consume the source stream to completion before returning a replay stream. The legacy design describes preserved non-blocking streaming, so the pending ruling keeps that discrepancy explicit.

1.7 End-to-end walkthrough

A traced LLM call crosses the public boundary, event bus, selected handlers, and the host's exporter.

SEQUENCE one traced operation



The handler attaches the new span to the current process before the wrapped operation runs. Nested instrumentation across different event types therefore inherits the active span, and ordinary child completion restores its parent before the outer operation continues. Same-type nesting and cleanup after callback failures are pending in issue 10.

Glossary

Every term the layer declares, with the context that owns it.

AgentObs event

owned by root

A backend-neutral telemetry event whose name contains an event prefix, an operation type, and a lifecycle phase.

Event prefix

owned by root

The configurable atom list prepended to every AgentObs event name and used by handlers when attaching subscriptions.

Handler

owned by root

A supervised telemetry subscriber that turns AgentObs events into records for an observability backend.

Instrumentation helper

owned by root

A public wrapper that executes an application operation while emitting its AgentObs event lifecycle.

Jido tracer

owned by root

The optional Jido observability implementation that maps composer lifecycle callbacks directly to OpenTelemetry spans.

OpenInference attributes

owned by root

The AI-observability semantic fields used to describe agent, LLM, tool, chain, and prompt spans.

ReqLLM adapter

owned by root

The optional integration that wraps ReqLLM calls and derives AgentObs metadata from their normalized responses.

Span context

owned by root

The OpenTelemetry trace state that identifies a span and its parent relationship in the emitting process.